Lecture 16

Dijkstra, Flow Networks

Source: Introduction to Algorithms, CLRS and Kleinberg & Tardos

Dijkstra's Algorithm: Sketch

Maintain a set of explored vertices S for which algorithm has found $d[u] = \delta(s, u)$:

Step 1: Initialise $S = \{s\}$, $d[s] = 0$.

Step 2: Choose an unexplored vertex v from $V \backslash S$ which minimizes:

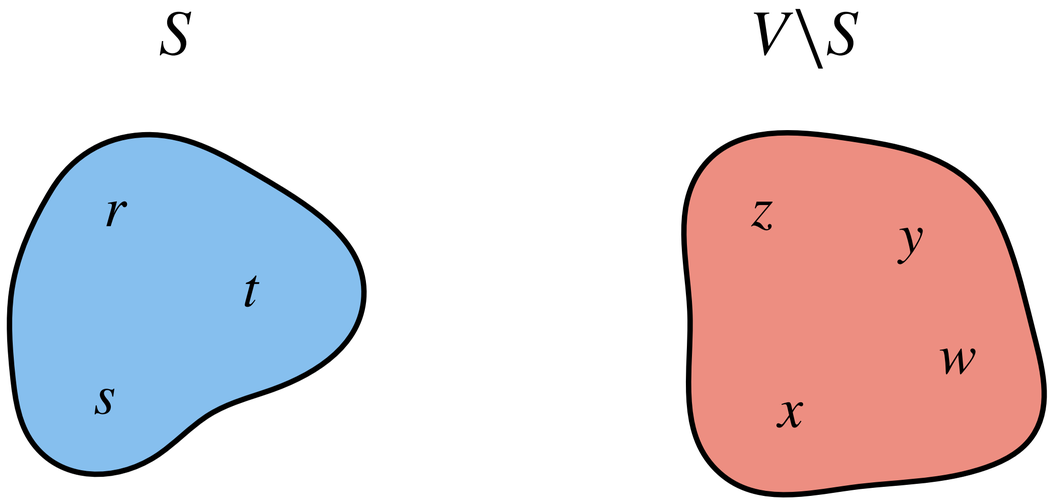
$$\pi[v] = \min_{(u,v) \in E,\ u \in S} d[u] + w(u, v)$$

Add v to S and set $d[v] = \Pi[v]$.

Step 3: Go to **Step 2** if it can be performed.

Dijkstra's Algorithm: Optimization

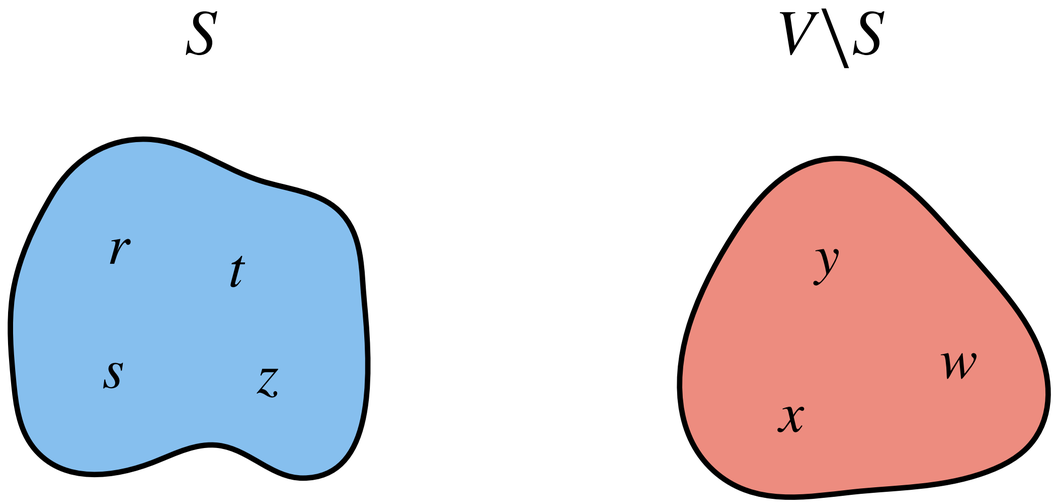
Computed $\pi[v]$ values of $V \backslash S$.



$d[s] = 0, d[r] = 3, d[t] = 2$

$\pi[z] = 4, \pi[y] = 8, \pi[x] = 6, \pi[w] =$ Invalid

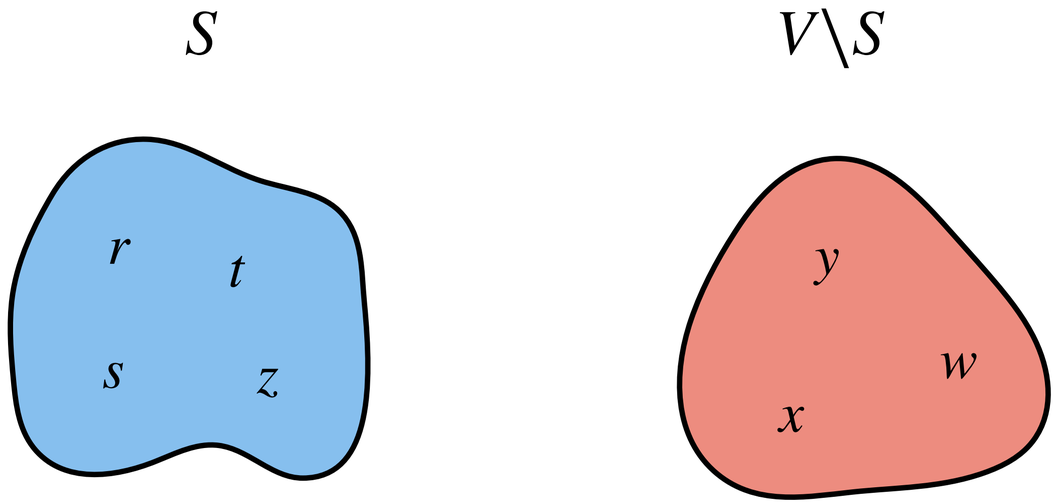
Dijkstra's Algorithm: Optimization



$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

Dijkstra's Algorithm: Optimization

Computing new $\pi[v]$ values after updating S.

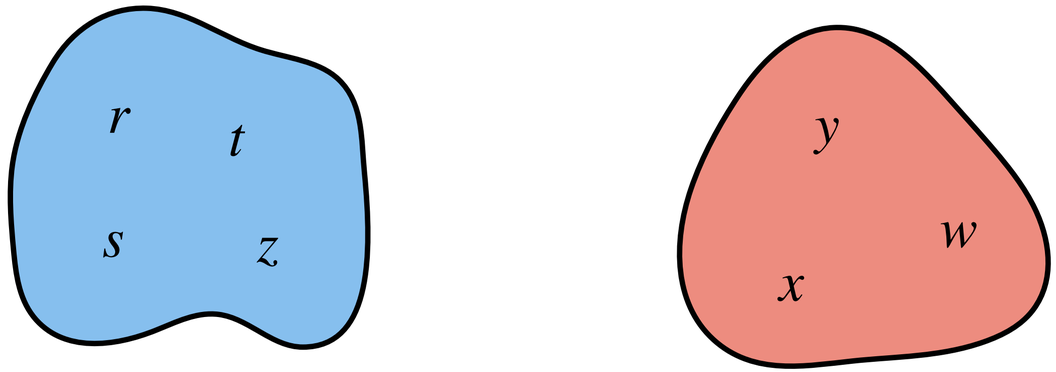


$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

Dijkstra's Algorithm: Optimization

Computing new $\pi[v]$ values after updating S.

$$\pi[v] = \min_{(u,v)\in E,\ u\in S} d[u] + w(u,v)$$

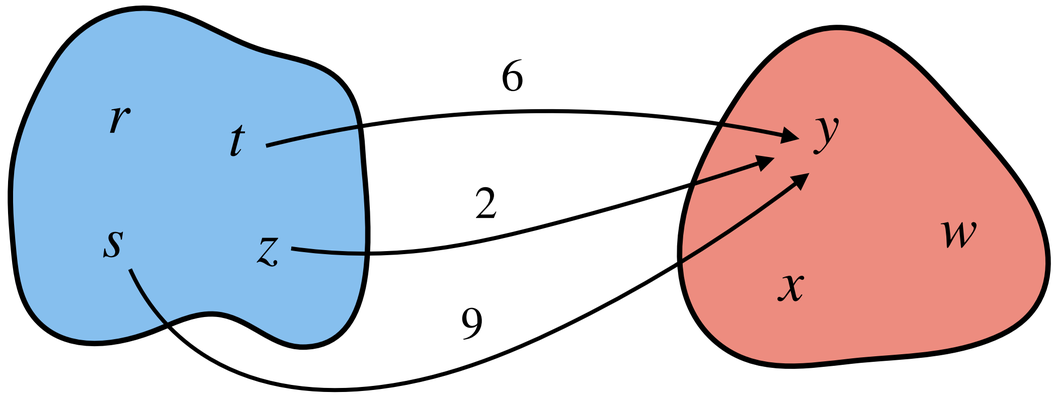


$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

Dijkstra's Algorithm: Optimization

Computing new $\pi[v]$ values after updating S.

$$\pi[v] = \min_{(u,v)\in E,\ u\in S} d[u] + w(u, v)$$

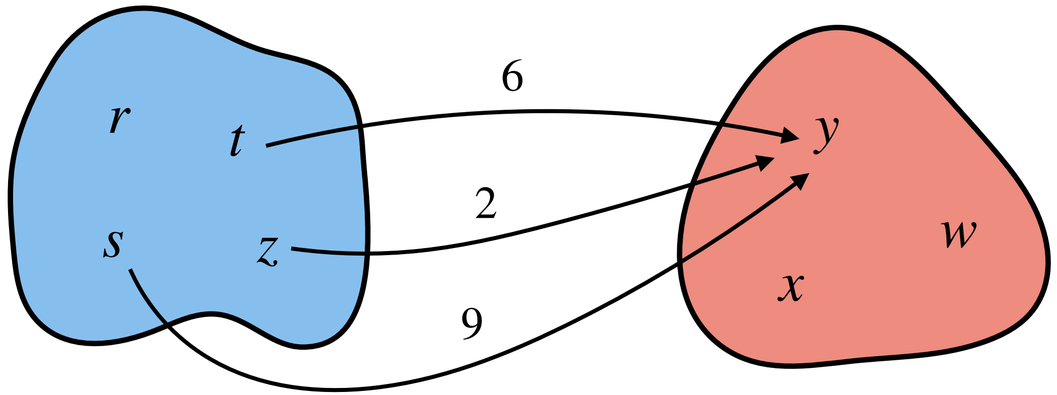


$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

Dijkstra's Algorithm: Optimization

Computing new $\pi[v]$ values after updating S.

$$\pi[v] = \min_{(u,v)\in E,\ u\in S} d[u] + w(u,v)$$



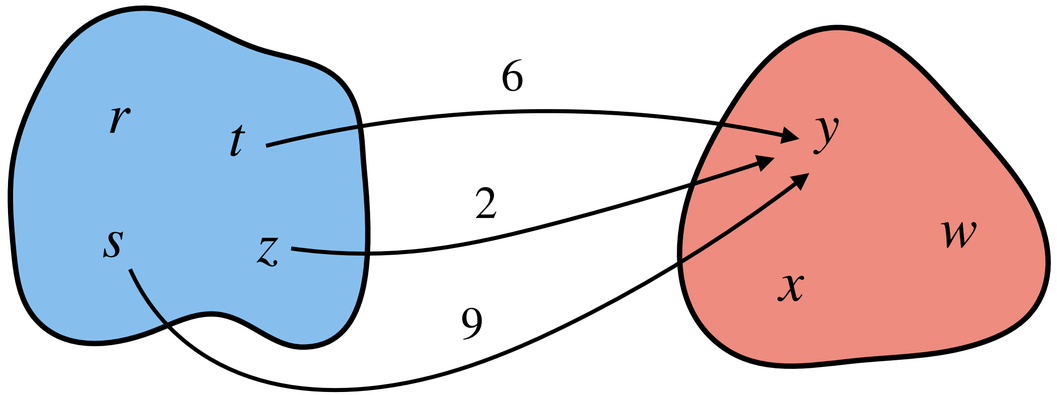
$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

$\pi[y]$ will be minimum of these values.

Dijkstra's Algorithm: Optimization

Computing new $\pi[v]$ values after updating S.

$$\pi[v] = \min_{(u,v)\in E,\ u\in S} d[u] + w(u,v)$$



$d[t] + w(t,y) = 8$

$d[s] + w(s,y) = 9$

$d[z] + w(z,y) = 6$

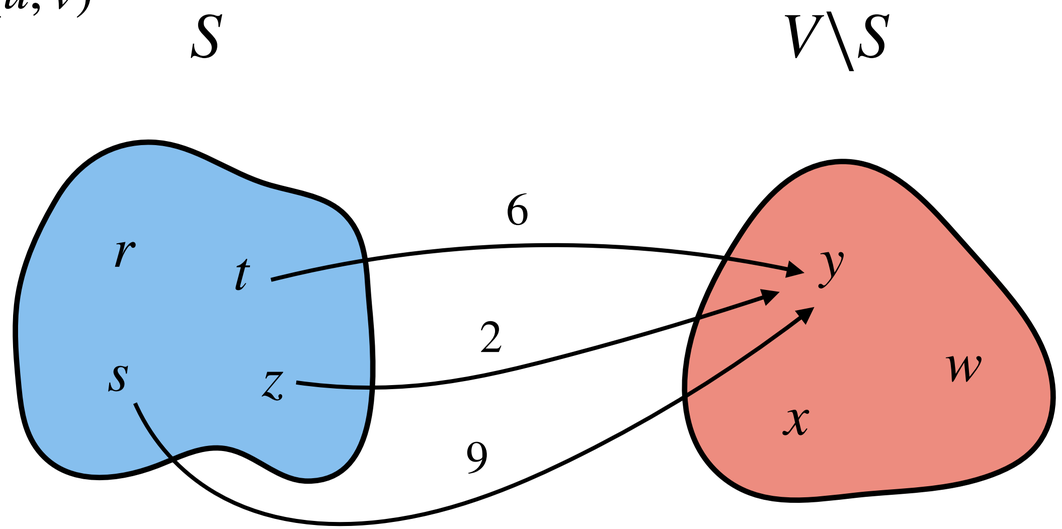
$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

$\pi[y]$ will be minimum of these values.

Dijkstra's Algorithm: Optimization

Computing new $\pi[v]$ values after updating S.

$$\pi[v] = \min_{(u,v)\in E,\ u\in S} d[u] + w(u,v)$$



Minimum of these is old $\pi[y]$

$d[t] + w(t,y) = 8$

$d[s] + w(s,y) = 9$

$d[z] + w(z,y) = 6$

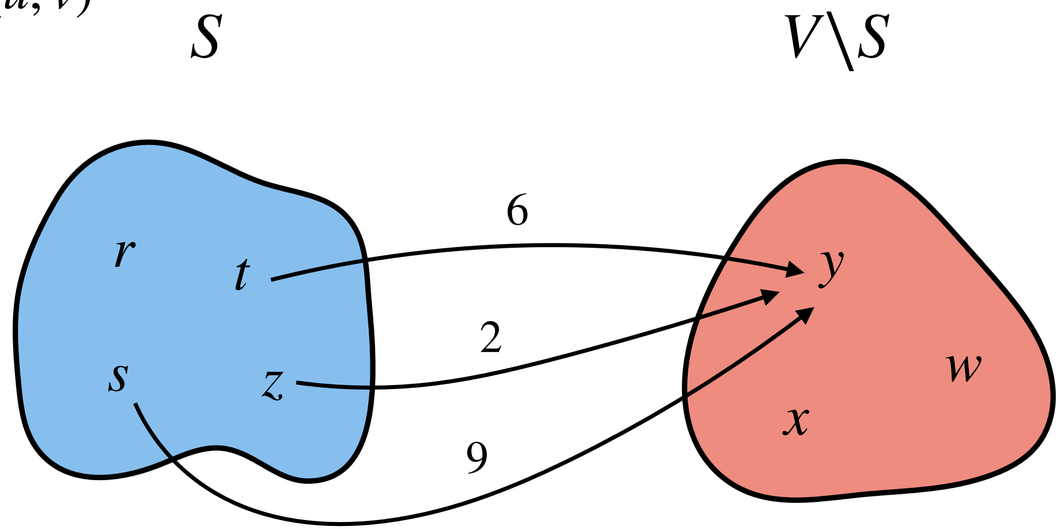
$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

$\pi[y]$ will be minimum of these values.

Dijkstra's Algorithm: Optimization

Computing new $\pi[v]$ values after updating S.

$$\pi[v] = \min_{(u,v)\in E,\ u\in S} d[u] + w(u,v)$$



Minimum of these is old $\pi[y]$

$d[t] + w(t,y) = 8$

$d[s] + w(s,y) = 9$

$d[z] + w(z,y) = 6$

$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

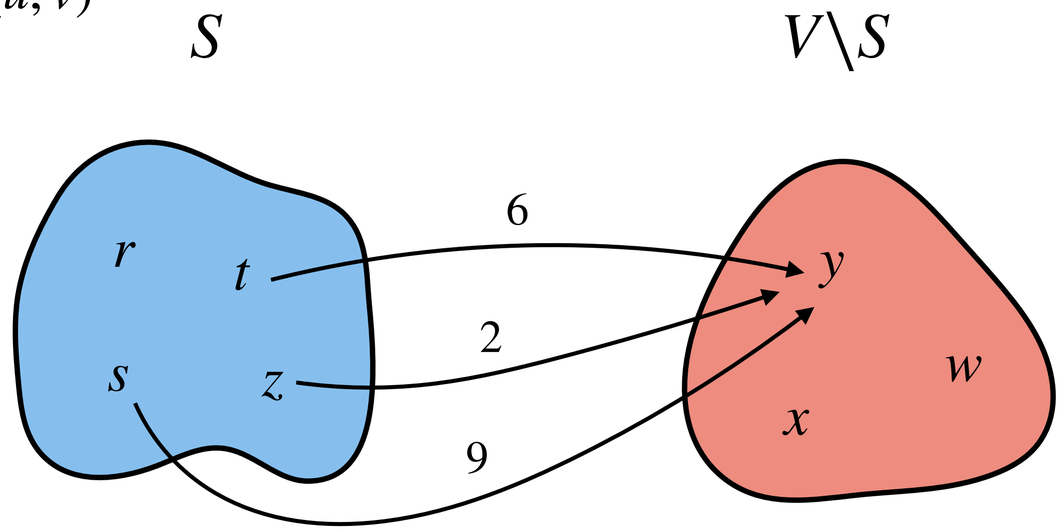
$\pi[y]$ will be minimum of these values.

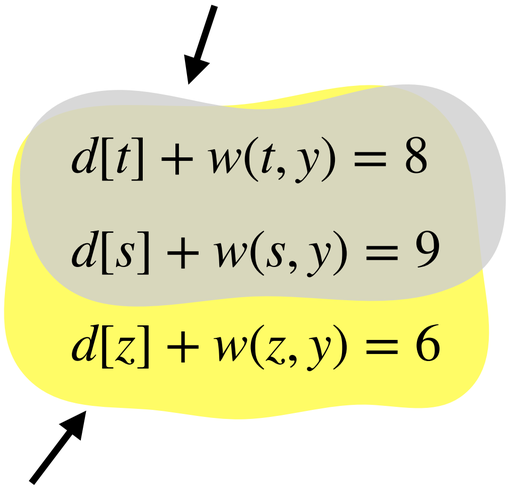
Observation: Let u be the vertex just added to S.

Dijkstra's Algorithm: Optimization

Computing new $\pi[v]$ values after updating S.

$$\pi[v] = \min_{(u,v)\in E,\ u\in S} d[u] + w(u,v)$$





$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

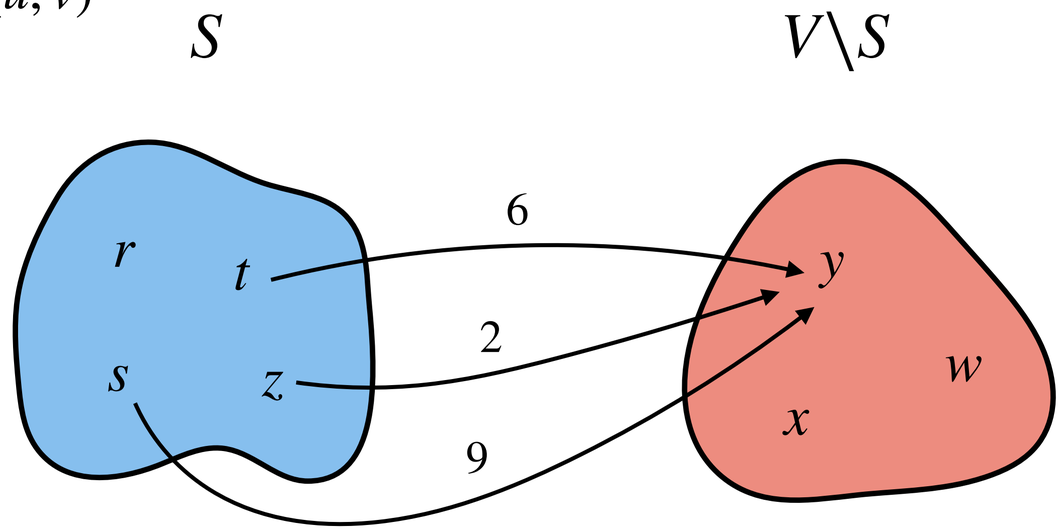
$\pi[y]$ will be minimum of these values.

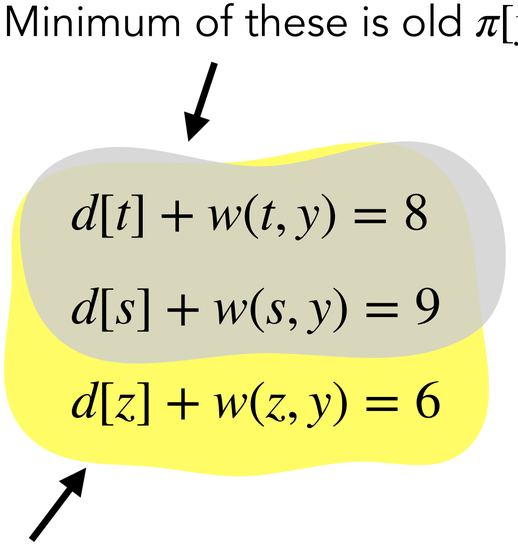
Observation: Let u be the vertex just added to S. Then, $\forall v \in V\backslash S$, such that (u, v) is an edge,

Dijkstra's Algorithm: Optimization

Computing new $\pi[v]$ values after updating S.

$$\pi[v] = \min_{(u,v)\in E,\ u\in S} d[u] + w(u,v)$$





$d[s] = 0, d[r] = 3, d[t] = 2, d[z] = 4$

$\pi[y]$ will be minimum of these values.

Observation: Let u be the vertex just added to S. Then, $\forall v \in V\backslash S$, such that (u, v) is an edge,

$$\pi[v] = \mathbf{Min}(\pi[v], d[u] + w(u,v))$$

Dijkstra’s Algorithm: Implementation

Dijkstra's Algorithm: Implementation

On $V \setminus S$ we are performing two operations:

Dijkstra's Algorithm: Implementation

On $V \setminus S$ we are performing two operations:

- Maintaining and updating π values.

Dijkstra's Algorithm: Implementation

On $V \setminus S$ we are performing two operations:

- Maintaining and updating π values.
- Removing the element with minimum π value.

Dijkstra's Algorithm: Implementation

On $V \setminus S$ we are performing two operations:

- Maintaining and updating π values.
- Removing the element with minimum π value.

What kind of data structure should we use for above operations?

Dijkstra's Algorithm: Implementation

On $V \setminus S$ we are performing two operations:

- Maintaining and updating π values.
- Removing the element with minimum π value.

What kind of data structure should we use for above operations?

Min-priority queue.

Dijkstra's Algorithm: Implementation

On $V \setminus S$ we are performing two operations:

- Maintaining and updating π values.
- Removing the element with minimum π value.

What kind of data structure should we use for above operations?

Min-priority queue. It can perform the following operations in $O(\log n)$ time.

Dijkstra's Algorithm: Implementation

On $V \setminus S$ we are performing two operations:

- Maintaining and updating π values.
- Removing the element with minimum π value.

What kind of data structure should we use for above operations?

Min-priority queue. It can perform the following operations in $O(\log n)$ time.

- $\textbf{Insert}(Q, u)$: Inserts element element u with a key in Q

Dijkstra's Algorithm: Implementation

On $V \setminus S$ we are performing two operations:

- Maintaining and updating π values.
- Removing the element with minimum π value.

What kind of data structure should we use for above operations?

Min-priority queue. It can perform the following operations in $O(\log n)$ time.

- **Insert**(Q, u): Inserts element element u with a key in Q
- **Extract-Min**(Q): Removes the element with minimum key from Q.

Dijkstra's Algorithm: Implementation

On $V \setminus S$ we are performing two operations:

- Maintaining and updating π values.
- Removing the element with minimum π value.

What kind of data structure should we use for above operations?

Min-priority queue. It can perform the following operations in $O(\log n)$ time.

- **Insert**(Q, u): Inserts element element u with a key in Q
- **Extract-Min**(Q): Removes the element with minimum key from Q.
- **Decrease-Key**(Q, u, d): Decreases the key of u to d.

Dijkstra's Algorithm: Implementation

On $V \setminus S$ we are performing two operations:

- Maintaining and updating π values.
- Removing the element with minimum π value.

What kind of data structure should we use for above operations?

Min-priority queue. It can perform the following operations in $O(\log n)$ time.

- **Insert**(Q, u): Inserts element element u with a key in Q
- **Extract-Min**(Q): Removes the element with minimum key from Q.
- **Decrease-Key**(Q, u, d): Decreases the key of u to d.

Conclusion: Form a min-priority queue of $V \setminus S$ where $keys$ are π values.

Dijkstra’s Algorithm: Pseudocode

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ // $S = \{u \mid u\text{'s distance is computed}\}$ and Q is a min-priority queue

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*
3. **for** each vertex $v \in V(G)$

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*
3. **for** each vertex $v \in V(G)$
4. $d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$ *// Q stores vertices with their π value as key.*

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*
3. **for** each vertex $v \in V(G)$
4. $d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$ *// Q stores vertices with their π value as key.*
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*
3. **for** each vertex $v \in V(G)$
4. $d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$ *// Q stores vertices with their π value as key.*
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*
3. **for** each vertex $v \in V(G)$
4. $d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$ *// Q stores vertices with their π value as key.*
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $u =$ **Extract-Min**(Q) *// u's distance is computed*

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*
3. **for** each vertex $v \in V(G)$
4. $d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$ *// Q stores vertices with their π value as key.*
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $u =$ **Extract-Min**(Q) *// u's distance is computed*
8. $S = S \cup \{u\}, d[u] = \pi[u]$ *// Add u to S and update its d*

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*
3. **for** each vertex $v \in V(G)$
4. $d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$ *// Q stores vertices with their π value as key.*
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $u =$ **Extract-Min**(Q) *// u's distance is computed*
8. $S = S \cup \{u\}, d[u] = \pi[u]$ *// Add u to S and update its d*
9. **for** each vertex $v \in Adj[u]$

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*
3. **for** each vertex $v \in V(G)$
4. $d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$ *// Q stores vertices with their π value as key.*
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $u =$ **Extract-Min**(Q) *// u's distance is computed*
8. $S = S \cup \{u\}, d[u] = \pi[u]$ *// Add u to S and update its d*
9. **for** each vertex $v \in Adj[u]$
10. **if** $\pi[v] > d[u] + w(u, v)$ *//Recall setting $\pi[v] =$ **Min**$(\pi[v], d[u] + w(u, v))$*

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*
3. **for** each vertex $v \in V(G)$
4. $d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$ *// Q stores vertices with their π value as key.*
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $u =$ **Extract-Min**(Q) *// u's distance is computed*
8. $S = S \cup \{u\}, d[u] = \pi[u]$ *// Add u to S and update its d*
9. **for** each vertex $v \in Adj[u]$
10. **if** $\pi[v] > d[u] + w(u, v)$ *//Recall setting $\pi[v] =$ **Min**$(\pi[v], d[u] + w(u, v))$*
11. $\pi[v] = d[u] + w(u, v)$

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$ *// $S = \{u \mid u$'s distance is computed$\}$ and Q is a min-priority queue*
2. Create array $d[1 : |V|], \pi[1 : |V|]$ *// for storing distance and π values*
3. **for** each vertex $v \in V(G)$
4. $d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$ *// Q stores vertices with their π value as key.*
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $u =$ **Extract-Min**(Q) *// u's distance is computed*
8. $S = S \cup \{u\}, d[u] = \pi[u]$ *// Add u to S and update its d*
9. **for** each vertex $v \in Adj[u]$
10. **if** $\pi[v] > d[u] + w(u, v)$ *//Recall setting $\pi[v] =$ **Min**$(\pi[v], d[u] + w(u, v))$*
11. $\pi[v] = d[u] + w(u, v)$
12. **Decrease-Key**$(Q, v, \pi[v])$

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

What if $v \in S$?

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty,$ **Insert**(Q, v)
5. $\pi[v] = 0,$ **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

What if $v \in S$? Then line 10 condition will be false as

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty,$ **Insert**(Q, v)
5. $\pi[v] = 0,$ **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

What if $v \in S$? Then line 10 condition will be false as $\pi[v]$ became $\delta(s, v)$ earlier and cannot further decrease.

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty,$ **Insert**(Q, v)
5. $\pi[v] = 0,$ **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

What happens when $\pi[u]$ is ∞?

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty,$ **Insert**(Q, v)
5. $\pi[v] = 0,$ **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

What happens when $\pi[u]$ is ∞?

$d[u]$ becoming ∞ is fine.

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty,$ **Insert**(Q, v)
5. $\pi[v] = 0,$ **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

What happens when $\pi[u]$ is ∞?

$d[u]$ becoming ∞ is fine. In line 10, $\infty + k$ is assumed to be ∞.

Dijkstra's Algorithm: Pseudocode

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty,$ **Insert**(Q, v)
5. $\pi[v] = 0,$ **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

What happens when $\pi[u]$ is ∞?

$d[u]$ becoming ∞ is fine. In line 10, $\infty + k$ is assumed to be ∞. Hence, condition of line 10 will be false.

Dijkstra's Algorithm: Analysis

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

Dijkstra's Algorithm: Analysis

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

Suppose G has n vertices and m edges.

Dijkstra's Algorithm: Analysis

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$

Dijkstra's Algorithm: Analysis

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty,$ **Insert**(Q, v)
5. $\pi[v] = 0,$ **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$

Dijkstra's Algorithm: Analysis

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$

Cost of this loop is $O(n \log n + m \log n)$

Dijkstra's Algorithm: Analysis

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty,$ **Insert**(Q, v)
5. $\pi[v] = 0,$ **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$

Cost of this loop is $O(n \log n + m \log n)$

(Every vertex is dequeued at most once, and when dequeued its adjacency list is traversed.)

Dijkstra's Algorithm: Analysis

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$

Cost of this loop is $O(n \log n + m \log n)$

(Every vertex is dequeued at most once, and when dequeued its adjacency list is traversed.)

Time complexity $= O(n \log n + m \log n)$

Dijkstra's Algorithm: Analysis

Dijkstra(G, s)**:**

1. $S = \emptyset, Q = \emptyset$
2. Create array $d[1 : |V|], \pi[1 : |V|]$
3. **for** each vertex $v \in V(G)$
4. $\quad d[v] = \infty, \pi[v] = \infty$, **Insert**$(Q, v)$
5. $\pi[v] = 0$, **Decrease-Key**$(Q, s, 0)$
6. **while** $Q \neq \emptyset$
7. $\quad u =$ **Extract-Min**(Q)
8. $\quad S = S \cup \{u\}, d[u] = \pi[u]$
9. $\quad$ **for** each vertex $v \in Adj[u]$
10. $\quad\quad$ **if** $\pi[v] > d[u] + w(u, v)$
11. $\quad\quad\quad \pi[v] = d[u] + w(u, v)$
12. $\quad\quad\quad$ **Decrease-Key**$(Q, v, \pi[v])$

Suppose G has n vertices and m edges.

Cost of this loop is $O(n)$

Cost of this loop is $O(n \log n + m \log n)$

(Every vertex is dequeued at most once, and when dequeued its adjacency list is traversed.)

Time complexity $= O(m \log n)$, when $m > n$

Dijkstra’s Algorithm: Correctness

Dijkstra's Algorithm: Correctness

Theorem: Dijkstra's algorithm, run on weighted, directed graph $G = (V, E)$ with

Dijkstra's Algorithm: Correctness

Theorem: Dijkstra's algorithm, run on weighted, directed graph $G = (V, E)$ with non-negative weight functions w and source vertex s, terminates with $d[u] = \delta(s, u)$ for all

Dijkstra's Algorithm: Correctness

Theorem: Dijkstra's algorithm, run on weighted, directed graph $G = (V, E)$ with non-negative weight functions w and source vertex s, terminates with $d[u] = \delta(s, u)$ for all vertices $u \in V$.

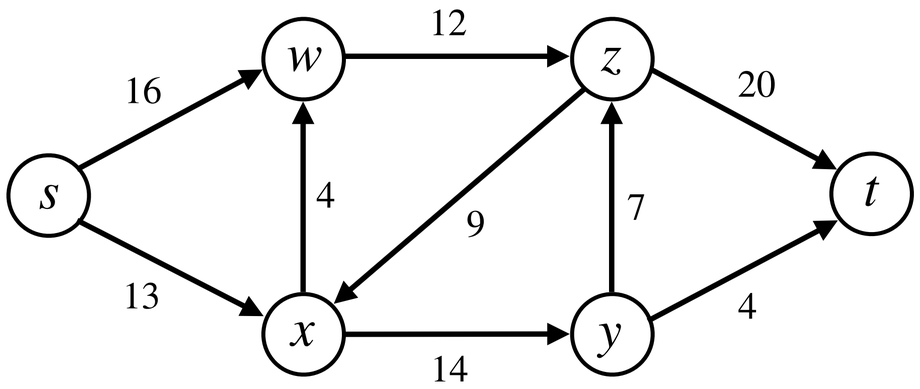
Dijkstra's Algorithm: Correctness

Theorem: Dijkstra's algorithm, run on weighted, directed graph $G = (V, E)$ with non-negative weight functions w and source vertex s, terminates with $d[u] = \delta(s, u)$ for all vertices $u \in V$.

Proof: Already done.

Flow Networks

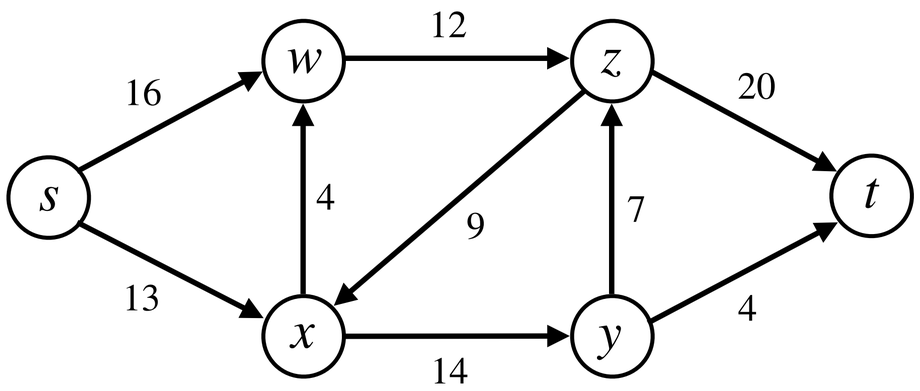
Flow Networks



(a)

Flow Networks

Figure (a) is flow network of a shipping company, where:

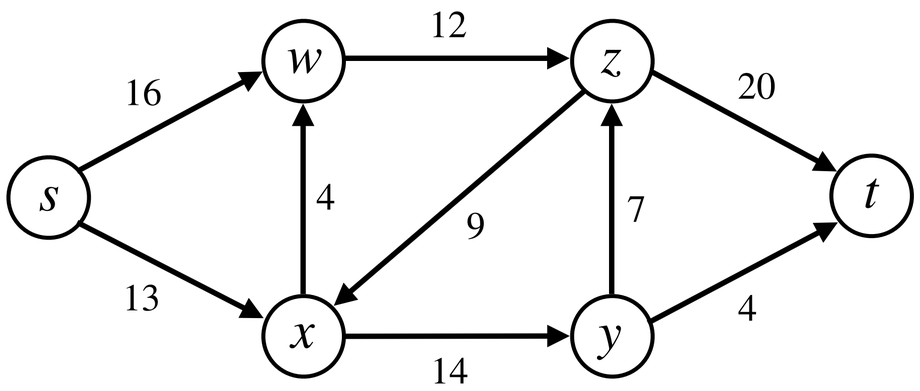


(a)

Flow Networks

Figure (a) is flow network of a shipping company, where:

- Vertices represent **cities**. s & t are the **source** & **sink** cities.

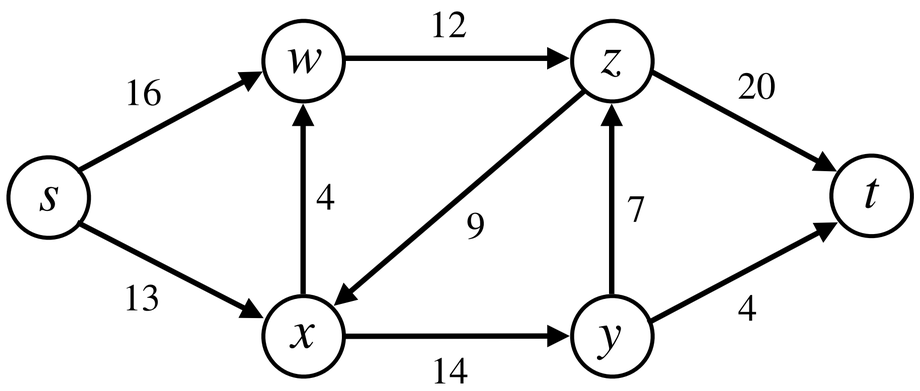


(a)

Flow Networks

Figure (a) is flow network of a shipping company, where:

- Vertices represent **cities**. s & t are the **source** & **sink** cities.
- The number on any (u, v) edge is the **maximum number of packets** that can go from u to v per day.



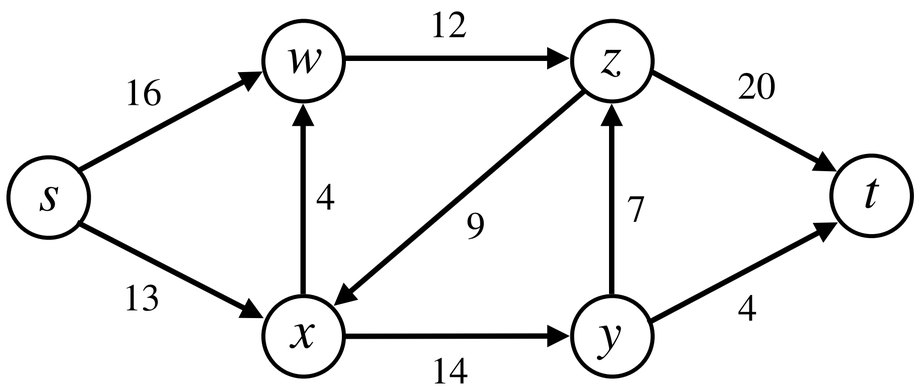
(a)

Flow Networks

Figure (a) is flow network of a shipping company, where:

- Vertices represent **cities**. s & t are the **source** & **sink** cities.
- The number on any (u, v) edge is the **maximum number of packets** that can go from u to v per day.

Goal: Find the **maximum number of packets** that can be shipped from s in a day if the packets



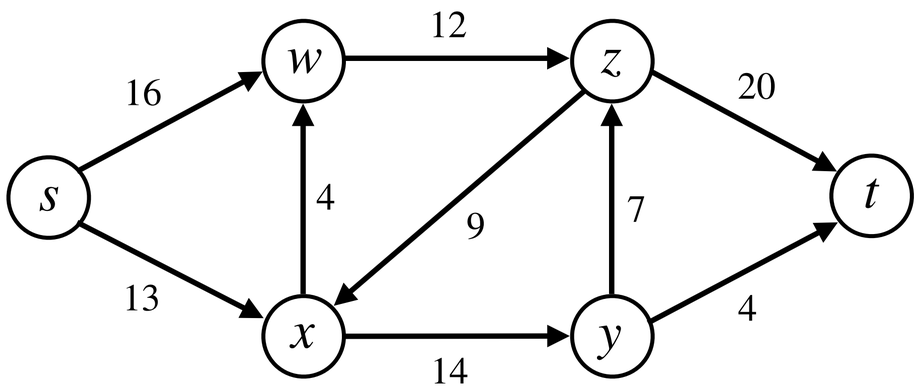
(a)

Flow Networks

Figure (a) is flow network of a shipping company, where:

- Vertices represent **cities**. s & t are the **source** & **sink** cities.
- The number on any (u, v) edge is the **maximum number of packets** that can go from u to v per day.

Goal: Find the **maximum number of packets** that can be shipped from s in a day if the packets received and sent by intermediate cities are equal in numbers.



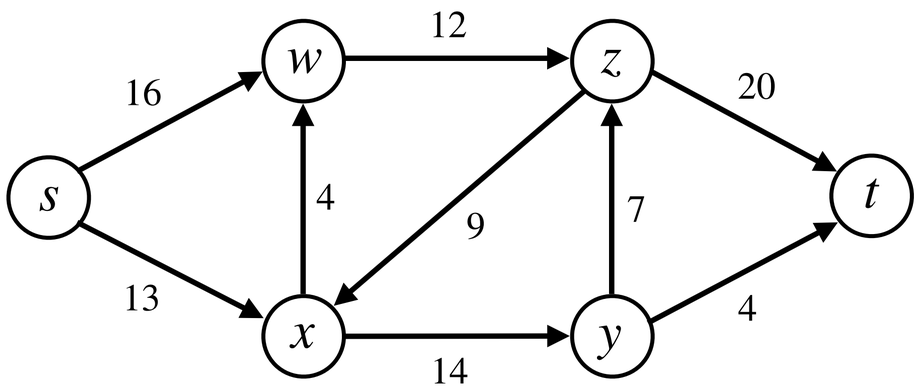
(a)

Flow Networks

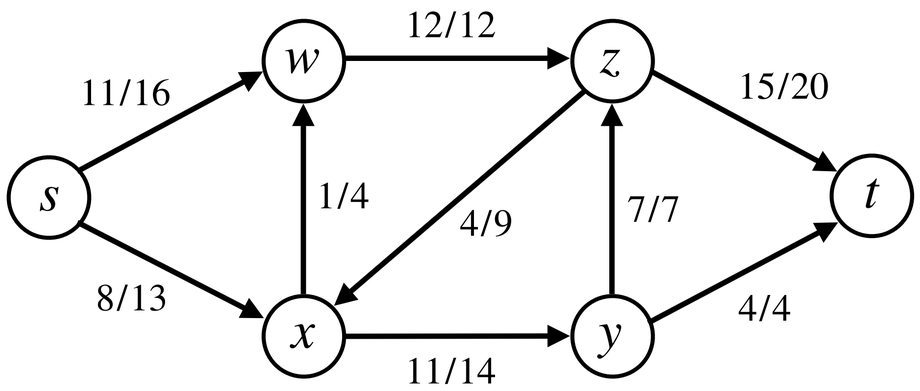
Figure (a) is flow network of a shipping company, where:

- Vertices represent **cities**. s & t are the **source** & **sink** cities.
- The number on any (u, v) edge is the **maximum number of packets** that can go from u to v per day.

Goal: Find the **maximum number of packets** that can be shipped from s in a day if the packets received and sent by intermediate cities are equal in numbers.



(a)



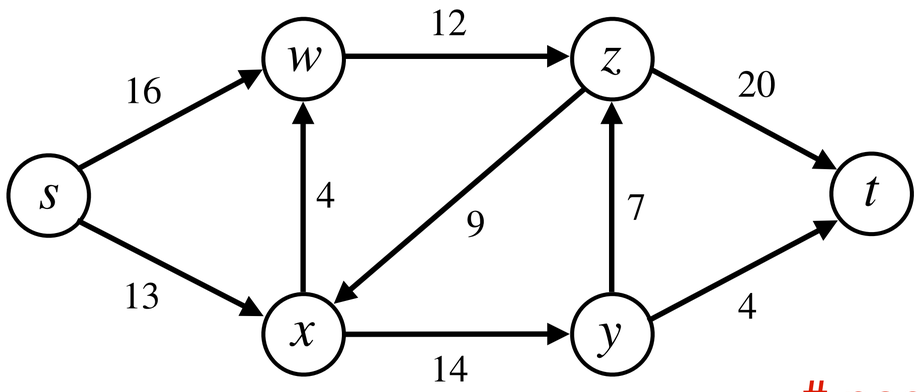
(b)

Flow Networks

Figure (a) is flow network of a shipping company, where:

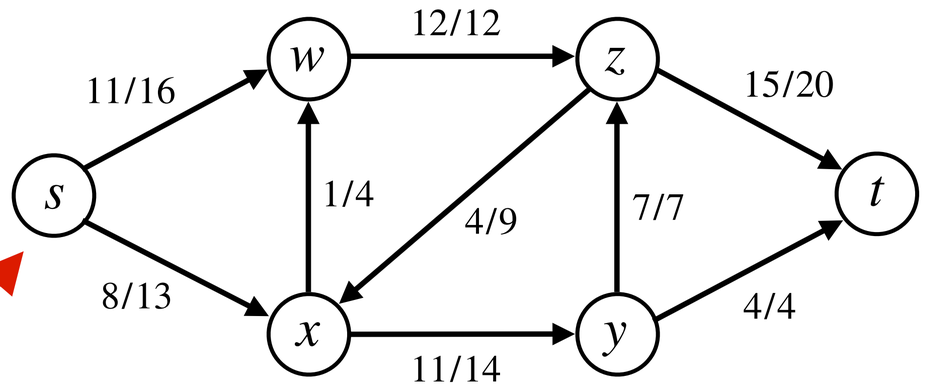
- Vertices represent **cities**. s & t are the **source** & **sink** cities.
- The number on any (u, v) edge is the **maximum number of packets** that can go from u to v per day.

Goal: Find the **maximum number of packets** that can be shipped from s in a day if the packets received and sent by intermediate cities are equal in numbers.



(a)

packets = 19



(b)

Flow Networks

Flow Networks

Defn: A flow network $G = (V, E)$ is a directed graph in which:

Flow Networks

Defn: A flow network $G = (V, E)$ is a directed graph in which:

- Each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$.

Flow Networks

Defn: A flow network $G = (V, E)$ is a directed graph in which:

- Each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$.
- If $(u, v) \in E$, then $(v, u) \notin E$.

Flow Networks

Defn: A flow network $G = (V, E)$ is a directed graph in which:

- Each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$.
- If $(u, v) \in E$, then $(v, u) \notin E$. (Reason will become clear soon.)

Flow Networks

Defn: A flow network $G = (V, E)$ is a directed graph in which:

- Each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$.
- If $(u, v) \in E$, then $(v, u) \notin E$. (Reason will become clear soon.)
- If $(u, v) \notin E$, we define $c(u, v) = 0$. No self-loops are present.

Flow Networks

Defn: A flow network $G = (V, E)$ is a directed graph in which:

- Each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$.
- If $(u, v) \in E$, then $(v, u) \notin E$. (Reason will become clear soon.)
- If $(u, v) \notin E$, we define $c(u, v) = 0$. No self-loops are present.
- Two distinguished vertices: source s (no incoming edges) and sink t (no outgoing edges).

Flow Networks

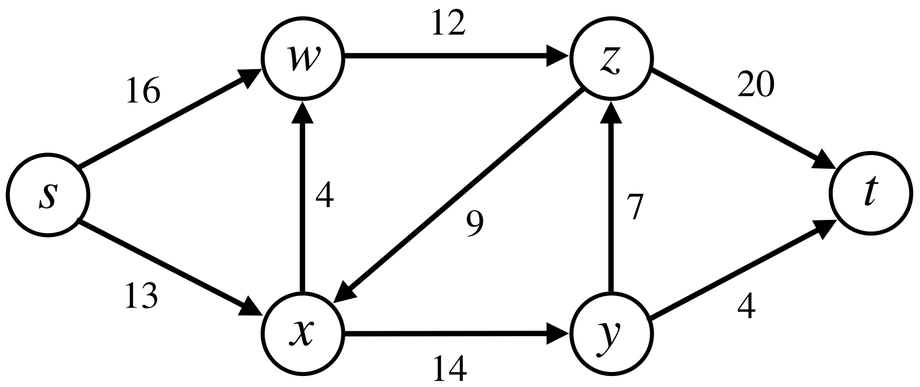
Defn: A flow network $G = (V, E)$ is a directed graph in which:

- Each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$.
- If $(u, v) \in E$, then $(v, u) \notin E$. (Reason will become clear soon.)
- If $(u, v) \notin E$, we define $c(u, v) = 0$. No self-loops are present.
- Two distinguished vertices: source s (no incoming edges) and sink t (no outgoing edges).
- For every $v \in V$, some $s \rightsquigarrow v \rightsquigarrow t$ path exists. Hence, $|E| \geq |V| - 1$.

Flow Networks

Defn: A flow network $G = (V, E)$ is a directed graph in which:

- Each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$.
- If $(u, v) \in E$, then $(v, u) \notin E$. (Reason will become clear soon.)
- If $(u, v) \notin E$, we define $c(u, v) = 0$. No self-loops are present.
- Two distinguished vertices: source s (no incoming edges) and sink t (no outgoing edges).
- For every $v \in V$, some $s \rightsquigarrow v \rightsquigarrow t$ path exists. Hence, $|E| \geq |V| - 1$.



Flows

Flows

Defn: Let $G = (V, E)$ be flow network with a capacity function c and source s and sink t.

Flows

Defn: Let $G = (V, E)$ be flow network with a capacity function c and source s and sink t.

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies the following two properties:

Flows

Defn: Let $G = (V, E)$ be flow network with a capacity function c and source s and sink t.

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies the following two properties:

- **Capacity constraint**: For all $u, v \in V$,

Flows

Defn: Let $G = (V, E)$ be flow network with a capacity function c and source s and sink t.

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies the following two properties:

- **Capacity constraint**: For all $u, v \in V$, $0 \leq f(u, v) \leq c(u, v)$.

Flows

Defn: Let $G = (V, E)$ be flow network with a capacity function c and source s and sink t.

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies the following two properties:

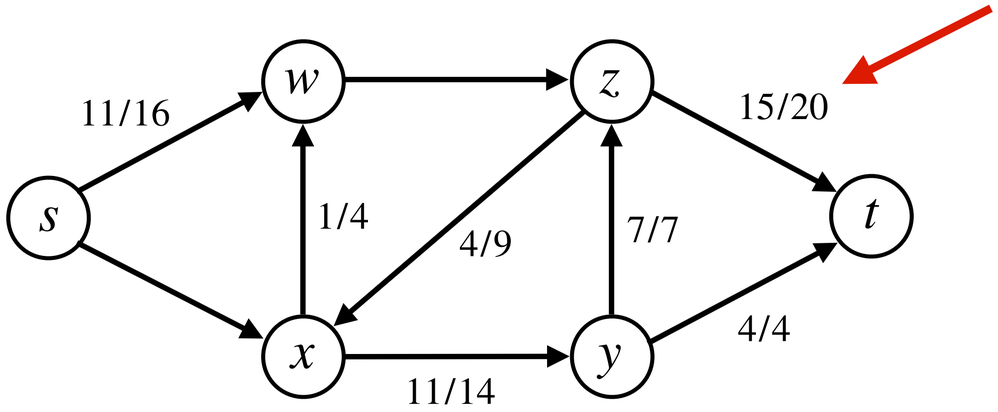
- **Capacity constraint**: For all $u, v \in V$, $0 \leq f(u, v) \leq c(u, v)$.
- **Flow conservation:** For all $u \in V \backslash \{s, t\}$,

Flows

Defn: Let $G = (V, E)$ be flow network with a capacity function c and source s and sink t.

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies the following two properties:

- **Capacity constraint**: For all $u, v \in V$, $0 \leq f(u, v) \leq c(u, v)$.
- **Flow conservation:** For all $u \in V \backslash \{s, t\}$, $\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$.

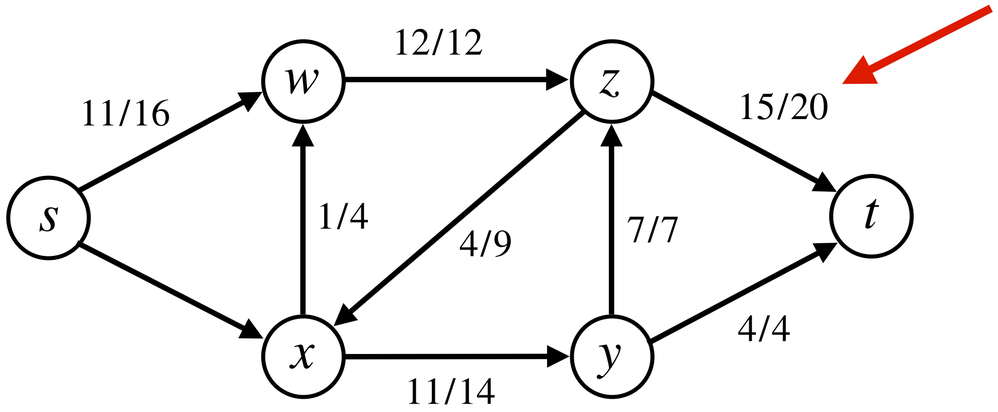


Flows

Defn: Let $G = (V, E)$ be flow network with a capacity function c and source s and sink t.

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies the following two properties:

- **Capacity constraint**: For all $u, v \in V$, $0 \leq f(u, v) \leq c(u, v)$.
- **Flow conservation:** For all $u \in V \backslash \{s, t\}$, $\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$.

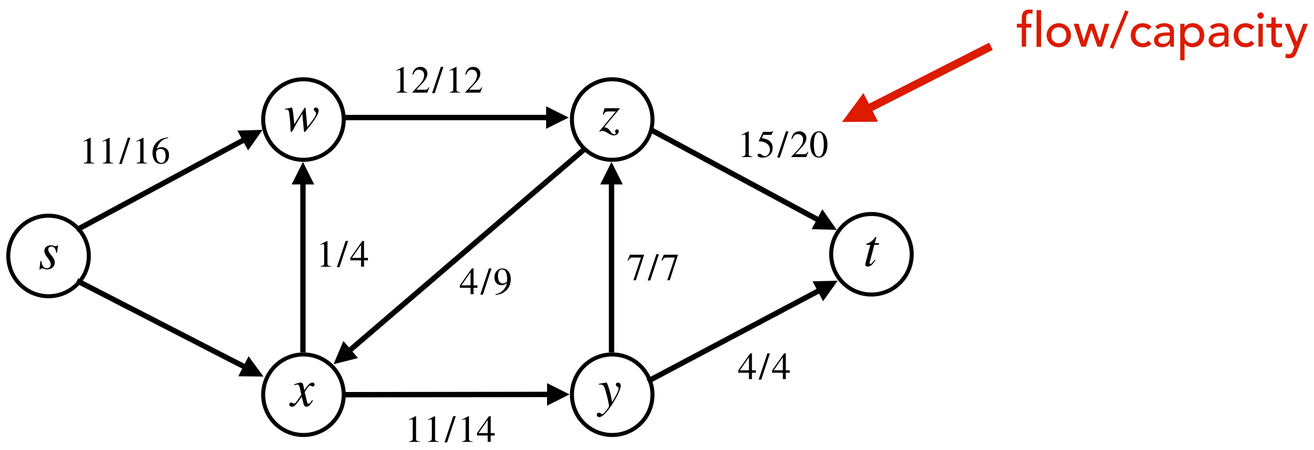


Flows

Defn: Let $G = (V, E)$ be flow network with a capacity function c and source s and sink t.

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies the following two properties:

- **Capacity constraint**: For all $u, v \in V$, $0 \leq f(u, v) \leq c(u, v)$.
- **Flow conservation:** For all $u \in V \backslash \{s, t\}$, $\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$.



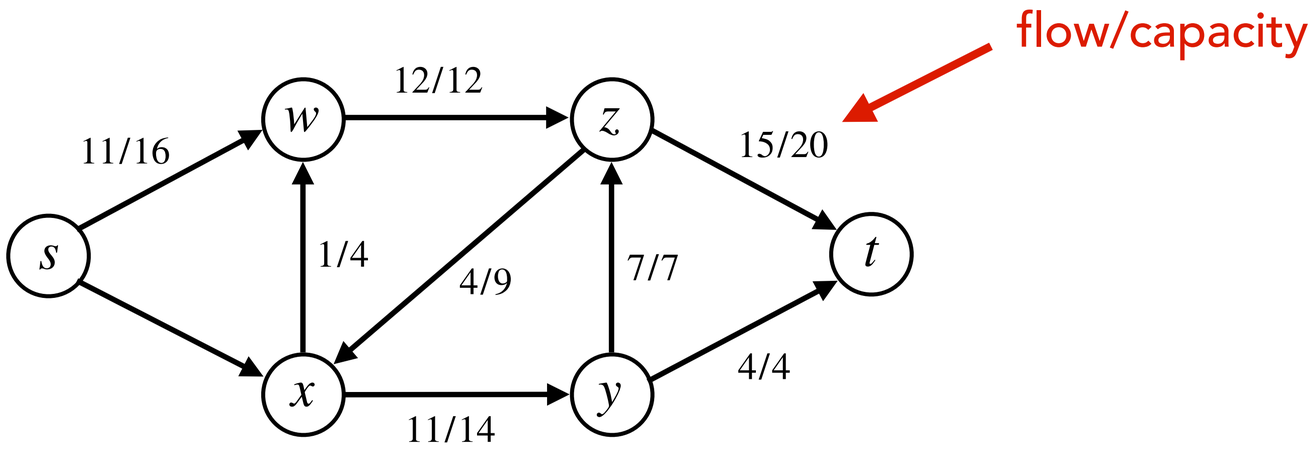
Flows

Defn: Let $G = (V, E)$ be flow network with a capacity function c and source s and sink t.

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies the following two properties:

- **Capacity constraint**: For all $u, v \in V$, $0 \leq f(u, v) \leq c(u, v)$.
- **Flow conservation:** For all $u \in V \backslash \{s, t\}$, $\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$.

Example:



Flows

Flows

Defn: Value $|f|$ of flow f is defined as flow out of s, i.e.,

Flows

Defn: Value $|f|$ of flow f is defined as flow out of s, i.e., $|f| = \sum_{v \in V} f(s, v)$.

Flows

Defn: Value $|f|$ of flow f is defined as flow out of s, i.e., $|f| = \sum_{v \in V} f(s, v)$.

Maxflow:

Flows

Defn: Value $|f|$ of flow f is defined as flow out of s, i.e., $|f| = \sum_{v \in V} f(s, v)$.

Maxflow:

Input: A flow network G with source s and sink t.

Flows

Defn: Value $|f|$ of flow f is defined as flow out of s, i.e., $|f| = \sum_{v \in V} f(s, v)$.

Maxflow:

Input: A flow network G with source s and sink t.

Output: Find a flow of maximum value.